

Module 11.2 - Advanced Routing

Learning Objectives: Master child routes and nested routing patterns, implement route parameters (static, dynamic, snapshot, paramMap), work with query parameters and fragments, and perform programmatic navigation using the Router service.

1. Child Routes and Nested Routing

Note: This module covers both module-based and standalone approaches to advanced routing. Examples marked with **(Module-Based)** or **(Standalone)** show the respective syntax. Standalone components (Angular 14+) offer simpler syntax and are recommended for new projects.

1.1 Understanding Child Routes

Child routes allow you to create nested routing structures where components can have their own set of routes. This is essential for building complex applications with hierarchical navigation.

Why Use Child Routes?

- Create nested navigation structures
- Keep related routes organized
- Share layout components across multiple child routes
- Build dashboard-style applications with sub-sections

1.2 Basic Child Routes Configuration

Module-Based Approach

Parent Component (dashboard.component.ts - Module-Based):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-dashboard',
  template: `
    <div class="dashboard">
      <h2>Dashboard</h2>

      <nav class="dashboard-nav">
        <a routerLink="overview" routerLinkActive="active">Overview</a>
        <a routerLink="analytics" routerLinkActive="active">Analytics</a>
        <a routerLink="reports" routerLinkActive="active">Reports</a>
      </nav>

      <!-- Child routes will render here -->
      <div class="dashboard-content">
        <router-outlet></router-outlet>
      </div>
    </div>
  `,
  styles: [`
    .dashboard {
      padding: 20px;
    }
    .dashboard-nav {
      display: flex;
      gap: 16px;
      margin: 20px 0;
      border-bottom: 1px solid #ddd;
      padding-bottom: 10px;
    }
    .dashboard-nav a {
      padding: 8px 16px;
      text-decoration: none;
      color: #333;
    }
    .dashboard-nav a.active {
      color: #007bff;
      border-bottom: 2px solid #007bff;
    }
    .dashboard-content {
      margin-top: 20px;
      padding: 20px;
      background: #f9f9f9;
      border-radius: 4px;
    }
  `]
})
```

```
`]  
})  
export class DashboardComponent {}
```

Child Components:

```
// overview.component.ts  
@Component({  
  selector: 'app-overview',  
  template: `  
    <h3>Dashboard Overview</h3>  
    <p>Welcome to the dashboard overview section.</p>  
    <div class="stats">  
      <div class="stat-card">  
        <h4>Total Users</h4>  
        <p>1,234</p>  
      </div>  
      <div class="stat-card">  
        <h4>Revenue</h4>  
        <p>$45,678</p>  
      </div>  
    </div>  
  `,  
})  
export class OverviewComponent {}  
  
// analytics.component.ts  
@Component({  
  selector: 'app-analytics',  
  template: `  
    <h3>Analytics</h3>  
    <p>View detailed analytics and metrics.</p>  
  `,  
})  
export class AnalyticsComponent {}  
  
// reports.component.ts  
@Component({  
  selector: 'app-reports',  
  template: `  
    <h3>Reports</h3>  
    <p>Generate and view reports.</p>  
  `
```

```
})  
export class ReportsComponent {}
```

Routing Configuration (app-routing.module.ts):

```
import { NgModule } from '@angular/core';  
import { RouterModule, Routes } from '@angular/router';  
import { DashboardComponent } from '../dashboard/dashboard.component';  
import { OverviewComponent } from '../dashboard/overview/overview.component';  
import { AnalyticsComponent } from '../dashboard/analytics/analytics.component';  
import { ReportsComponent } from '../dashboard/reports/reports.component';  
  
const routes: Routes = [  
  {  
    path: 'dashboard',  
    component: DashboardComponent,  
    children: [  
      // Child routes  
      { path: 'overview', component: OverviewComponent },  
      { path: 'analytics', component: AnalyticsComponent },  
      { path: 'reports', component: ReportsComponent },  
  
      // Default child route  
      { path: '', redirectTo: 'overview', pathMatch: 'full' }  
    ]  
  }  
];  
  
@NgModule({  
  imports: [RouterModule.forRoot(routes)],  
  exports: [RouterModule]  
})  
export class AppRoutingModule {}
```

Standalone Approach

Parent Component (dashboard.component.ts - Standalone):

```
import { Component } from '@angular/core';  
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';
```

```
@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive], // ← Import routing directives
  template: `
    <div class="dashboard">
      <h2>Dashboard</h2>
      <nav>
        <a routerLink="overview" routerLinkActive="active">Overview</a>
        <a routerLink="analytics" routerLinkActive="active">Analytics</a>
        <a routerLink="reports" routerLinkActive="active">Reports</a>
      </nav>
      <div class="content">
        <router-outlet></router-outlet>
      </div>
    </div>
  `,
  styles: [`
    .dashboard nav {
      margin: 20px 0;
    }
    .dashboard nav a {
      margin-right: 15px;
      padding: 8px 16px;
      text-decoration: none;
      color: #333;
      border: 1px solid #ddd;
      border-radius: 4px;
    }
    .dashboard nav a.active {
      background-color: #007bff;
      color: white;
      border-color: #007bff;
    }
  `]
})
export class DashboardComponent {}
```

Child Components (Standalone):

```
// overview.component.ts (Standalone)
import { Component } from '@angular/core';

@Component({
  selector: 'app-overview',
```

```
standalone: true,
template: `
  <div>
    <h3>Dashboard Overview</h3>
    <p>Total Users: 1,234</p>
    <p>Revenue: $45,678</p>
  </div>
`
})
export class OverviewComponent {}

// analytics.component.ts (Standalone)
import { Component } from '@angular/core';

@Component({
  selector: 'app-analytics',
  standalone: true,
  template: `<h3>Analytics Data</h3>`
})
export class AnalyticsComponent {}

// reports.component.ts (Standalone)
import { Component } from '@angular/core';

@Component({
  selector: 'app-reports',
  standalone: true,
  template: `<h3>Reports</h3>`
})
export class ReportsComponent {}
```

Routing Configuration (app.routes.ts - Standalone):

```
import { Routes } from '@angular/router';
import { DashboardComponent } from '../dashboard/dashboard.component';
import { OverviewComponent } from '../dashboard/overview/overview.component';
import { AnalyticsComponent } from '../dashboard/analytics/analytics.component';
import { ReportsComponent } from '../dashboard/reports/reports.component';

export const routes: Routes = [
  {
    path: 'dashboard',
    component: DashboardComponent,
    children: [
      { path: '', redirectTo: 'overview', pathMatch: 'full' },
```

```
{ path: 'overview', component: OverviewComponent },
{ path: 'analytics', component: AnalyticsComponent },
{ path: 'reports', component: ReportsComponent }
]
}
];
```

Key Differences in Standalone:

- No NgModule or RouterModule needed
- Import RouterOutlet, RouterLink, RouterLinkActive directly in component
- Routes are simple TypeScript exports
- Use `standalone: true` in component decorator

URL Structure:

/dashboard → redirects to → /dashboard/overview
 /dashboard/overview → Shows Overview component
 /dashboard/analytics → Shows Analytics component
 /dashboard/reports → Shows Reports component

Rendered Structure:

```
┌ Dashboard Component ───────────┐
│ [Dashboard Header] │
│ [Overview | Analytics | Reports] │
│ ┌ router-outlet ───────────┐ │
│ │ [Overview Component Content] │ │
│ │ - Total Users: 1,234 │ │
│ │ - Revenue: $45,678 │ │
│ └──────────────────┘ │
└──────────────────┘
```

1.3 Nested Child Routes (Multi-Level)

You can nest child routes at multiple levels.

Deep Nested Routing Configuration:

```
const routes: Routes = [
  {
    path: 'admin',
```

```

component: AdminComponent,
children: [
  {
    path: 'users',
    component: UsersComponent,
    children: [
      { path: '', component: UserListComponent },
      { path: 'new', component: UserCreateComponent },
      { path: ':id', component: UserDetailComponent },
      { path: ':id/edit', component: UserEditComponent }
    ]
  },
  {
    path: 'settings',
    component: SettingsComponent,
    children: [
      { path: 'general', component: GeneralSettingsComponent },
      { path: 'security', component: SecuritySettingsComponent },
      { path: 'notifications', component: NotificationSettingsComponent }
    ]
  }
]
};

// URL Examples:
// /admin/users → UserListComponent
// /admin/users/new → UserCreateComponent
// /admin/users/123 → UserDetailComponent (id = 123)
// /admin/users/123/edit → UserEditComponent
// /admin/settings/general → GeneralSettingsComponent

```

Admin Component Template:

```

<div class="admin-layout">
  <aside class="sidebar">
    <nav>
      <a routerLink="users" routerLinkActive="active">Users</a>
      <a routerLink="settings" routerLinkActive="active">Settings</a>
    </nav>
  </aside>

  <main class="content">
    <!-- First level child routes -->

```

```

    <router-outlet></router-outlet>
  </main>
</div>

```

Users Component Template:

```

<div class="users-section">
  <h2>User Management</h2>

  <nav class="sub-nav">
    <a routerLink="." routerLinkActive="active" [routerLinkActiveOptions]="{exact: true}">
      List
    </a>
    <a routerLink="new" routerLinkActive="active">Create New</a>
  </nav>

  <!-- Second level child routes -->
  <router-outlet></router-outlet>
</div>

```

Nested Router Outlets:

Level 1: AdminComponent has <router-outlet>

↓

Level 2: UsersComponent has <router-outlet>

↓

Level 3: UserDetailComponent (no outlet)

URL: /admin/users/123

```

└─ AdminComponent ───────────────────┘
│ [Sidebar: Users | Settings] │
│ └─ UsersComponent ─────────────────┘ │
│ │ [Sub-nav: List | Create New] │ │
│ │ └─ UserDetailComponent ─────────┘ │ │
│ │ │ User ID: 123 │ │ │
│ │ │ Name: John Doe │ │ │
│ │ │ Email: john@example.com │ │ │
│ │ └──────────────────────────┘ │ │
│ └──────────────────────────┘ │
└──────────────────────────┘

```

1.4 Relative Navigation in Child Routes

Component TypeScript (Works for both Module-Based and Standalone):

```
import { Component } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';

// For standalone, add: standalone: true, imports: [RouterLink]
@Component({
  selector: 'app-user-list',
  template: `
    <h3>User List</h3>
    <div *ngFor="let user of users">
      <p>{{ user.name }}</p>

      <!-- Relative navigation in template -->
      <a [routerLink]="[user.id]">View Details</a>
      <a [routerLink]="[user.id, 'edit']">Edit</a>
    </div>

    <button (click)="navigateToNew()">Create New User</button>
  `
})
export class UserListComponent {
  users = [
    { id: 1, name: 'John Doe' },
    { id: 2, name: 'Jane Smith' },
    { id: 3, name: 'Bob Johnson' }
  ];

  constructor(
    private router: Router,
    private route: ActivatedRoute
  ) {}

  navigateToNew() {
    // Relative navigation - relative to current route
    this.router.navigate(['new'], { relativeTo: this.route });

    // Current route: /admin/users
    // Navigates to: /admin/users/new
  }

  navigateToUser(userId: number) {
    // Navigate to sibling route
  }
}
```

```

    this.router.navigate(['../', userId], { relativeTo: this.route });
  }

  navigateUp() {
    // Navigate to parent route
    this.router.navigate(['../'], { relativeTo: this.route });
  }
}

```

Relative Navigation Paths:

- `./segment` or `segment` - Navigate to child
- `../segment` - Navigate to sibling (go up one level, then down)
- `../../segment` - Navigate up two levels
- `../` - Navigate to parent route

2. Route Parameters

2.1 Static vs Dynamic Parameters

Type	Definition	Example	Use Case
Static	Fixed path segments	<code>/users/list</code>	Known routes
Dynamic	Variable path segments	<code>/users/:id</code>	Resource identifiers
Query Params	Optional parameters	<code>/users?sort=name&page=2</code>	Filters, pagination
Fragment	Hash navigation	<code>/users#section-1</code>	Scroll to element

2.2 Dynamic Route Parameters

Route Configuration:

```
const routes: Routes = [  
  { path: 'users/:id', component: UserDetailComponent },  
  { path: 'users/:id/posts/:postId', component: UserPostComponent },  
  { path: 'products/:category/:productId', component: ProductDetailComponent }  
];
```

Accessing Parameters - Snapshot Method (Module-Based):

```
import { Component, OnInit } from '@angular/core';  
import { ActivatedRoute } from '@angular/router';  
  
@Component({  
  selector: 'app-user-detail',  
  template: `  
    <div class="user-detail">  
      <h2>User Details</h2>  
      <p>User ID: {{ userId }}</p>  
      <p>Name: {{ user?.name }}</p>  
      <p>Email: {{ user?.email }}</p>  
    </div>  
  `,  
})  
export class UserDetailComponent implements OnInit {  
  userId: string | null = null;  
  user: any = null;  
  
  constructor(private route: ActivatedRoute) {}  
  
  ngOnInit() {  
    // Using snapshot - synchronous, one-time read  
    this.userId = this.route.snapshot.paramMap.get('id');  
    console.log('User ID from snapshot:', this.userId);  
  
    // Load user data  
    if (this.userId) {  
      this.loadUser(this.userId);  
    }  
  }  
  
  loadUser(id: string) {  
    // Simulate loading user data  
    this.user = {  
      id: id,  
      name: 'John Doe',  
    };  
  }  
}
```

```
    email: 'john@example.com'
  };
}
}
```

URL: /users/123

User Details

User ID: 123

Name: John Doe

Email: john@example.com

Console Output:

User ID from snapshot: 123

Accessing Parameters - Snapshot Method (Standalone):

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-user-detail',
  standalone: true, // ← Standalone component
  template: `
    <div>
      <h2>User Details</h2>
      <p>User ID: {{ userId }}</p>
      <p>Name: {{ user?.name }}</p>
      <p>Email: {{ user?.email }}</p>
    </div>
  `,
})
export class UserDetailComponent implements OnInit {
  userId: string | null = null;
  user: any = null;

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    // Read parameter from snapshot - ONE TIME ONLY
    this.userId = this.route.snapshot.paramMap.get('id');
    console.log('User ID from snapshot:', this.userId);
  }
}
```

```
// Load user data
this.loadUser(this.userId);
}

loadUser(id: string | null) {
  // Simulate loading user data
  this.user = {
    id: id,
    name: 'John Doe',
    email: 'john@example.com'
  };
}
}
```

Snapshot Limitation: The `snapshot` approach only reads parameters once during component initialization. If you navigate from `/users/1` to `/users/2` while staying in the same component, the snapshot won't update because Angular reuses the component instance.

2.3 Observable Route Parameters (paramMap)

Using paramMap Observable - Reactive Approach (Works for Both Module-Based and Standalone):

The paramMap observable approach is the same for both module-based and standalone components. For standalone, simply add `standalone: true` to the decorator and import any template directives needed.

Example:

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

@Component({
  selector: 'app-user-detail',
  template: `
    <div class="user-detail">
```

```

    <h2>User Details</h2>
    <p>User ID: {{ userId }}</p>
    <p>Name: {{ user?.name }}</p>

    <nav>
      <a [routerLink]="['/users', 1]">User 1</a>
      <a [routerLink]="['/users', 2]">User 2</a>
      <a [routerLink]="['/users', 3]">User 3</a>
    </nav>
  </div>
  `
})
export class UserDetailComponent implements OnInit, OnDestroy {
  userId: string | null = null;
  user: any = null;
  private destroy$ = new Subject<void>();

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    // Using paramMap observable - reactive, updates on param changes
    this.route.paramMap.pipe(
      takeUntil(this.destroy$)
    ).subscribe(params => {
      this.userId = params.get('id');
      console.log('User ID changed to:', this.userId);

      if (this.userId) {
        this.loadUser(this.userId);
      }
    });
  }

  loadUser(id: string) {
    console.log('Loading user:', id);
    this.user = {
      id: id,
      name: `User ${id}`,
      email: `user${id}@example.com`
    };
  }

  ngOnDestroy() {
    this.destroy$.next();
  }
}

```

```
    this.destroy$.complete();  
  }  
}
```

Initial Load: /users/1

Console: User ID changed to: 1

Console: Loading user: 1

Display: User 1 (user1@example.com)

User clicks "User 2" link

URL changes to: /users/2

Console: User ID changed to: 2

Console: Loading user: 2

Display: User 2 (user2@example.com)

User clicks "User 3" link

URL changes to: /users/3

Console: User ID changed to: 3

Console: Loading user: 3

Display: User 3 (user3@example.com)

✓ Component reacts to parameter changes without re-initialization!

2.4 Multiple Route Parameters

Route Configuration:

```
const routes: Routes = [  
  {  
    path: 'products/:category/:productId',  
    component: ProductDetailComponent  
  }  
];
```

Component:

```
@Component({  
  selector: 'app-product-detail',  
  template: `
```

```

<div>
  <h2>Product Details</h2>
  <p>Category: {{ category }}</p>
  <p>Product ID: {{ productId }}</p>
  <p>Product Name: {{ product?.name }}</p>
  <p>Price: {{ product?.price | currency }}</p>
</div>
,
}))
export class ProductDetailComponent implements OnInit {
  category: string | null = null;
  productId: string | null = null;
  product: any = null;

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    this.route.paramMap.subscribe(params => {
      // Extract multiple parameters
      this.category = params.get('category');
      this.productId = params.get('productId');

      console.log('Category:', this.category);
      console.log('Product ID:', this.productId);

      if (this.category && this.productId) {
        this.loadProduct(this.category, this.productId);
      }
    });
  }

  loadProduct(category: string, id: string) {
    // Simulate loading product
    this.product = {
      id: id,
      category: category,
      name: `Product ${id}`,
      price: 99.99
    };
  }
}

```

URL: /products/electronics/12345

Product Details

Category: electronics
Product ID: 12345
Product Name: Product 12345
Price: \$99.99

Console Output:

Category: electronics
Product ID: 12345

2.5 ParamMap API Methods

Complete ParamMap Example:

```
this.route.paramMap.subscribe(params => {  
  // Get single parameter  
  const id = params.get('id');  
  console.log('ID:', id); // "123" or null  
  
  // Get all parameter keys  
  const keys = params.keys;  
  console.log('Parameter keys:', keys); // ["id", "category"]  
  
  // Check if parameter exists  
  const hasId = params.has('id');  
  console.log('Has ID:', hasId); // true or false  
  
  // Get all values (for parameters that can appear multiple times)  
  const allIds = params.getAll('id');  
  console.log('All IDs:', allIds); // ["123"]  
  
  // Get all parameters as object  
  const allParams: any = {};  
  params.keys.forEach(key => {  
    allParams[key] = params.get(key);  
  });  
  console.log('All parameters:', allParams);  
  // { id: "123", category: "electronics" }  
});
```

3. Query Parameters

3.1 Understanding Query Parameters

Query parameters are optional parameters appended to the URL after a `?` symbol. They're commonly used for:

- Filtering and sorting lists
- Pagination
- Search terms
- Optional settings that don't affect route matching

Query Parameter Syntax:

```
// Single parameter
/users?sort=name

// Multiple parameters
/users?sort=name&order=asc&page=2

// Array parameters
/products?tags=electronics&tags=sale&tags=new
```

3.2 Passing Query Parameters

The syntax for passing query parameters is identical in templates for both approaches. However, standalone components must import the RouterLink directive.

In Template with routerLink (Both Approaches):

```
<!-- Object syntax -->
<a [routerLink]="['/users']"
  [queryParams]="{sort: 'name', order: 'asc'}">
  Sort by Name
</a>

<!-- Multiple parameters -->
<a [routerLink]="['/products']"
  [queryParams]="{category: 'electronics', minPrice: 100, maxPrice: 500}">
```

```
Electronics $100-$500
</a>

<!-- Preserve existing query params -->
<a [routerLink]="['/users']"
  [queryParams]="{page: 2}"
  queryParamsHandling="merge">
  Next Page
</a>
```

Programmatically with Router (Same for Both Approaches):

The Router service and its methods work identically in both module-based and standalone components. The only difference is that standalone components need `standalone: true` and must import any template directives.

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
// For standalone: add RouterLink to imports if used in template

@Component({
  selector: 'app-user-list',
  template: `
    <button (click)="filterByRole('admin')">Show Admins</button>
    <button (click)="sortBy('name')">Sort by Name</button>
    <button (click)="goToPage(2)">Page 2</button>
  `
})
export class UserListComponent {
  constructor(private router: Router) {}

  filterByRole(role: string) {
    this.router.navigate(['/users'], {
      queryParams: { role: role }
    });
    // Navigates to: /users?role=admin
  }

  sortBy(field: string) {
    this.router.navigate(['/users'], {
      queryParams: { sort: field, order: 'asc' }
    });
  }
}
```

```
// Navigates to: /users?sort=name&order=asc
}

goToPage(page: number) {
  this.router.navigate(['/users'], {
    queryParams: { page: page },
    queryParamsHandling: 'merge' // Keep existing query params
  });
  // If current URL is /users?sort=name
  // Navigates to: /users?sort=name&page=2
}
}
```

3.3 Reading Query Parameters

Using Snapshot (One-Time Read):

```
@Component({
  selector: 'app-user-list',
  template: `
    <h2>Users</h2>
    <p>Sort: {{ sortField }}</p>
    <p>Order: {{ sortOrder }}</p>
    <p>Page: {{ currentPage }}</p>
  `
})
export class UserListComponent implements OnInit {
  sortField: string | null = null;
  sortOrder: string | null = null;
  currentPage: number = 1;

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    // Read from snapshot
    const queryParams = this.route.snapshot.queryParamMap;

    this.sortField = queryParams.get('sort');
    this.sortOrder = queryParams.get('order');

    const pageParam = queryParams.get('page');
    this.currentPage = pageParam ? parseInt(pageParam) : 1;

    console.log('Query params:', {
```

```
        sort: this.sortField,  
        order: this.sortOrder,  
        page: this.currentPage  
    });  
}  
}
```

URL: /users?sort=name&order=asc&page=2

Users

Sort: name

Order: asc

Page: 2

Console Output:

Query params: {sort: "name", order: "asc", page: 2}

Using Observable (Reactive Updates):

```
@Component({  
  selector: 'app-product-list',  
  template: `  
    <div class="filters">  
      <button (click)="setCategory('electronics')">Electronics</button>  
      <button (click)="setCategory('books')">Books</button>  
      <button (click)="setCategory('clothing')">Clothing</button>  
    </div>  
  
    <h3>{{ category || 'All' }} Products</h3>  
    <p>Showing page {{ page }} of results</p>  
    <div *ngFor="let product of products">  
      {{ product.name }}  
    </div>  
  `,  
})  
export class ProductListComponent implements OnInit, OnDestroy {  
  category: string | null = null;  
  page: number = 1;  
  products: any[] = [];  
  
  private destroy$ = new Subject<void>();  
  
  constructor(  

```

```
private route: ActivatedRoute,
private router: Router
) {}

ngOnInit() {
  // Subscribe to query parameter changes
  this.route.queryParamMap.pipe(
    takeUntil(this.destroy$)
  ).subscribe(params => {
    this.category = params.get('category');
    const pageParam = params.get('page');
    this.page = pageParam ? parseInt(pageParam) : 1;

    console.log('Query params updated:', {
      category: this.category,
      page: this.page
    });

    // Load products based on query params
    this.loadProducts();
  });
}

loadProducts() {
  console.log(`Loading ${this.category || 'all'} products, page ${this.page}`);

  // Simulate loading products
  this.products = [
    { name: `Product 1 (${this.category || 'all'})` },
    { name: `Product 2 (${this.category || 'all'})` },
    { name: `Product 3 (${this.category || 'all'})` }
  ];
}

setCategory(category: string) {
  this.router.navigate([], {
    relativeTo: this.route,
    queryParams: { category: category, page: 1 },
    queryParamsHandling: 'merge'
  });
}

ngOnDestroy() {
  this.destroy$.next();
}
```

```

    this.destroy$.complete();
  }
}

```

Initial URL: /products

All Products

Showing page 1 of results

- Product 1 (all)
- Product 2 (all)
- Product 3 (all)

User clicks "Electronics":

URL: /products?category=electronics&page=1

Console: Query params updated: {category: "electronics", page: 1}

Console: Loading electronics products, page 1

Electronics Products

Showing page 1 of results

- Product 1 (electronics)
- Product 2 (electronics)
- Product 3 (electronics)

3.4 Query Params Handling Strategies

Strategy	Description	Example
' ' (default)	Replace all query params	Old: ?a=1&b=2 → New: ?c=3
'merge'	Merge with existing params	Old: ?a=1&b=2 → New: ?a=1&b=2&c=3
'preserve'	Keep all existing params	Old: ?a=1&b=2 → New: ?a=1&b=2

Example Usage:

```

// Replace all query params
this.router.navigate(['/users'], {
  queryParams: { sort: 'name' }
});
// Current: /users?filter=admin&page=2

```

```
// Result: /users?sort=name

// Merge with existing
this.router.navigate(['/users'], {
  queryParams: { sort: 'name' },
  queryParamsHandling: 'merge'
});
// Current: /users?filter=admin&page=2
// Result: /users?filter=admin&page=2&sort=name

// Preserve existing
this.router.navigate(['/users'], {
  queryParamsHandling: 'preserve'
});
// Current: /users?filter=admin&page=2
// Result: /users?filter=admin&page=2
```

3.5 Real-World Example: Search and Filter

Product Search Component:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { Subject } from 'rxjs';
import { debounceTime, takeUntil } from 'rxjs/operators';

@Component({
  selector: 'app-product-search',
  template: `
    <div class="search-container">
      <h2>Product Search</h2>

      <input
        [(ngModel)]="searchTerm"
        (ngModelChange)="onSearchChange()"
        placeholder="Search products...">

      <div class="filters">
        <select [(ngModel)]="selectedCategory" (change)="applyFilters()">
          <option value="">All Categories</option>
          <option value="electronics">Electronics</option>
          <option value="books">Books</option>
          <option value="clothing">Clothing</option>
        </select>
      </div>
    </div>
```

```

    <select [(ngModel)]="sortBy" (change)="applyFilters()">
      <option value="name">Name</option>
      <option value="price">Price</option>
      <option value="rating">Rating</option>
    </select>

    <select [(ngModel)]="sortOrder" (change)="applyFilters()">
      <option value="asc">Ascending</option>
      <option value="desc">Descending</option>
    </select>
  </div>

  <div class="results">
    <p>Found {{ products.length }} products</p>
    <div *ngFor="let product of products" class="product-card">
      <h4>{{ product.name }}</h4>
      <p>{{ product.price | currency }}</p>
    </div>
  </div>

  <div class="pagination">
    <button (click)="goToPage(currentPage - 1)"
      [disabled]="currentPage === 1">
      Previous
    </button>
    <span>Page {{ currentPage }}</span>
    <button (click)="goToPage(currentPage + 1)">
      Next
    </button>
  </div>
</div>
,
})
export class ProductSearchComponent implements OnInit, OnDestroy {
  searchTerm = '';
  selectedCategory = '';
  sortBy = 'name';
  sortOrder = 'asc';
  currentPage = 1;
  products: any[] = [];

  private searchSubject = new Subject<string>();
  private destroy$ = new Subject<void>();

  constructor(
    private route: ActivatedRoute,

```

```
private router: Router
) {}

ngOnInit() {
  // Read initial query params
  this.route.queryParamMap.pipe(
    takeUntil(this.destroy$)
  ).subscribe(params => {
    this.searchTerm = params.get('search') || '';
    this.selectedCategory = params.get('category') || '';
    this.sortBy = params.get('sort') || 'name';
    this.sortOrder = params.get('order') || 'asc';
    this.currentPage = parseInt(params.get('page') || '1');

    this.loadProducts();
  });

  // Debounce search input
  this.searchSubject.pipe(
    debounceTime(300),
    takeUntil(this.destroy$)
  ).subscribe(term => {
    this.updateQueryParams();
  });
}

onSearchChange() {
  this.searchSubject.next(this.searchTerm);
}

applyFilters() {
  this.currentPage = 1; // Reset to first page
  this.updateQueryParams();
}

goToPage(page: number) {
  if (page < 1) return;
  this.currentPage = page;
  this.updateQueryParams();
}

updateQueryParams() {
  this.router.navigate([], {
    relativeTo: this.route,
    queryParams: {
      search: this.searchTerm || null,
      category: this.selectedCategory || null,
```

```
        sort: this.sortBy,
        order: this.sortOrder,
        page: this.currentPage
      },
      queryParamsHandling: 'merge'
    });
  }

  loadProducts() {
    console.log('Loading products with filters:', {
      search: this.searchTerm,
      category: this.selectedCategory,
      sort: this.sortBy,
      order: this.sortOrder,
      page: this.currentPage
    });

    // Simulate loading products
    this.products = [
      { name: 'Product A', price: 29.99 },
      { name: 'Product B', price: 49.99 },
      { name: 'Product C', price: 19.99 }
    ];
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
    this.searchSubject.complete();
  }
}
```

URL Evolution:

Initial: /products

User types "laptop":

→ /products?search=laptop&sort=name&order=asc&page=1

User selects "Electronics":

→ /products?search=laptop&category=electronics&sort=name&order=asc&page=1

User changes sort to "Price":

→ /products?search=laptop&category=electronics&sort=price&order=asc&page=1

User clicks "Next" page:

→ /products?search=laptop&category=electronics&sort=price&order=asc&page=2

✓ All filters preserved in URL - can bookmark or share!

4. URL Fragments

4.1 Understanding Fragments

URL fragments (also called hash fragments) are the part of the URL after the # symbol. They're commonly used for:

- Scrolling to specific sections on a page
- Deep linking within single-page sections
- Preserving scroll position

Fragment Syntax:

```
/article#introduction  
/docs#api-reference  
/profile#settings
```

4.2 Setting Fragments

In Template:

```
<nav>  
  <a [routerLink]="['/article']" fragment="introduction">Introduction</a>  
  <a [routerLink]="['/article']" fragment="examples">Examples</a>  
  <a [routerLink]="['/article']" fragment="conclusion">Conclusion</a>  
</nav>  
  
<!-- Generates: -->
```

```
<!-- /article#introduction -->
<!-- /article#examples -->
<!-- /article#conclusion -->
```

Programmatically:

```
export class ArticleComponent {
  constructor(private router: Router) {}

  scrollToSection(section: string) {
    this.router.navigate(['/article'], {
      fragment: section
    });
  }

  scrollToIntroduction() {
    this.scrollToSection('introduction');
    // Navigates to: /article#introduction
  }
}
```

4.3 Reading and Using Fragments

Component with Fragment Handling:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { ViewportScroller } from '@angular/common';

@Component({
  selector: 'app-article',
  template: `
    <div class="article">
      <nav class="table-of-contents">
        <a [routerLink]="[]" fragment="intro">Introduction</a>
        <a [routerLink]="[]" fragment="content">Main Content</a>
        <a [routerLink]="[]" fragment="conclusion">Conclusion</a>
      </nav>

      <section id="intro">
        <h2>Introduction</h2>
        <p>This is the introduction section...</p>
      </section>
    </div>
  `
})
export class ArticleComponent implements OnInit {
  constructor(
    private router: ActivatedRoute,
    private viewportScroller: ViewportScroller
  ) {}

  ngOnInit() {
    this.router.fragment.subscribe(fragment => {
      this.viewportScroller.scrollToAnchor(fragment);
    });
  }
}
```

```

</section>

<section id="content">
  <h2>Main Content</h2>
  <p>This is the main content section...</p>
</section>

<section id="conclusion">
  <h2>Conclusion</h2>
  <p>This is the conclusion section...</p>
</section>
</div>
`,
styles: [
  section {
    min-height: 400px;
    padding: 20px;
    margin: 20px 0;
    border: 1px solid #ddd;
  }
  .table-of-contents {
    position: sticky;
    top: 0;
    background: white;
    padding: 10px;
    border-bottom: 2px solid #007bff;
  }
  .table-of-contents a {
    margin-right: 20px;
  }
]
})
export class ArticleComponent implements OnInit {
  constructor(
    private route: ActivatedRoute,
    private viewportScroller: ViewportScroller
  ) {}

  ngOnInit() {
    // Listen to fragment changes
    this.route.fragment.subscribe(fragment => {
      if (fragment) {
        console.log('Scrolling to fragment:', fragment);

        // Scroll to element with matching id
        setTimeout(() => {
          this.viewportScroller.scrollToAnchor(fragment);

```

```
    }, 100);  
  }  
});  
}
```

User clicks "Main Content" link:

URL: /article#content

Console: Scrolling to fragment: content

Page scrolls to: <section id="content">

User clicks "Conclusion" link:

URL: /article#conclusion

Console: Scrolling to fragment: conclusion

Page scrolls to: <section id="conclusion">

4.4 Combining Fragments with Query Params

```
<a [routerLink]="['/docs']"  
  [queryParams]="{version: '2.0'}"  
  fragment="api-reference">  
  API Reference (v2.0)  
</a>  
  
<!-- Generates: /docs?version=2.0#api-reference -->
```

Programmatically:

```
this.router.navigate(['/docs'], {  
  queryParams: { version: '2.0', lang: 'en' },  
  fragment: 'api-reference'  
});  
  
// Navigates to: /docs?version=2.0&lang=en#api-reference
```

5. Programmatic Navigation

Universal Approach: The Router service and all its navigation methods work exactly the same way in both module-based and standalone components. The examples in this section apply to both approaches.

5.1 Router Service Overview

The `Router` service provides methods for programmatic navigation in your application.

Key Navigation Methods:

Method	Description	Use Case
<code>navigate()</code>	Navigate using commands array	Most common, flexible navigation
<code>navigateByUrl()</code>	Navigate using absolute URL string	Simple URL navigation
<code>createUrlTree()</code>	Create URL without navigating	Generate URLs for external use
<code>parseUrl()</code>	Parse URL string to <code>UrlTree</code>	URL manipulation

5.2 `navigate()` Method

Basic Navigation:

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';

@Component({
  selector: 'app-navigation-demo',
  template: `
    <button (click)="goToHome()">Go Home</button>
    <button (click)="goToUsers()">View Users</button>
    <button (click)="goToUserDetail(123)">View User 123</button>
    <button (click)="goToDashboard()">Go to Dashboard</button>
  `
})
```

```
export class NavigationDemoComponent {
  constructor(private router: Router) {}

  goToHome() {
    // Navigate to root
    this.router.navigate(['/']);
  }

  goToUsers() {
    // Navigate to /users
    this.router.navigate(['/users']);
  }

  goToUserDetail(userId: number) {
    // Navigate to /users/123
    this.router.navigate(['/users', userId]);
  }

  goToDashboard() {
    // Navigate to nested route
    this.router.navigate(['/dashboard', 'overview']);
    // Result: /dashboard/overview
  }
}
```

5.3 navigate() with NavigationExtras

Advanced Navigation Options:

```
import { Component } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-advanced-navigation',
  template: `
    <button (click)="navigateWithQueryParams()">With Query Params</button>
    <button (click)="navigateWithFragment()">With Fragment</button>
    <button (click)="navigateRelative()">Relative Navigation</button>
    <button (click)="navigateAndReplace()">Replace History</button>
  `
})
export class AdvancedNavigationComponent {
  constructor(
    private router: Router,
```

```
private route: ActivatedRoute
) {}

navigateWithQueryParams() {
  this.router.navigate(['/products'], {
    queryParams: {
      category: 'electronics',
      sort: 'price',
      order: 'asc'
    }
  });
  // Navigates to: /products?category=electronics&sort=price&order=asc
}

navigateWithFragment() {
  this.router.navigate(['/article'], {
    fragment: 'section-2'
  });
  // Navigates to: /article#section-2
}

navigateRelative() {
  // Navigate relative to current route
  this.router.navigate(['../sibling'], {
    relativeTo: this.route
  });

  // If current route is /parent/child
  // Navigates to: /parent/sibling
}

navigateAndReplace() {
  // Replace current history entry instead of adding new one
  this.router.navigate(['/login'], {
    replaceUrl: true
  });

  // User can't use "back" button to return to previous page
}

navigateWithState() {
  // Pass state data (not visible in URL)
  this.router.navigate(['/details'], {
    state: {
      userData: {
        name: 'John Doe',
        email: 'john@example.com'
      }
    }
  });
}
```

```
    }  
  }  
});  
}  
  
navigateCombined() {  
  // Combine multiple options  
  this.router.navigate(['/products', 'electronics', 123], {  
    queryParams: { view: 'detailed' },  
    fragment: 'reviews',  
    relativeTo: this.route,  
    queryParamsHandling: 'merge'  
  });  
  // Result: /products/electronics/123?view=detailed#reviews  
}  
}
```

5.4 navigateByUrl() Method

String-Based Navigation:

```
export class SimpleNavigationComponent {  
  constructor(private router: Router) {}  
  
  goToAbsoluteUrl() {  
    // Navigate using absolute URL string  
    this.router.navigateByUrl('/users/123');  
  }  
  
  goToUrlWithQuery() {  
    // Query params in URL string  
    this.router.navigateByUrl('/products?category=books&sort=price');  
  }  
  
  goToUrlWithFragment() {  
    // Fragment in URL string  
    this.router.navigateByUrl('/article#introduction');  
  }  
  
  goToComplex() {  
    // Complex URL  
    this.router.navigateByUrl('/products/electronics/456?view=detailed#specs');  
  }  
}
```

```
dynamicNavigation(path: string) {  
  // Navigate to dynamic URL  
  this.router.navigateByUrl(path);  
}  
}
```

navigate() vs navigateByUrl():

- **navigate():** More flexible, supports relative navigation, better for dynamic routes
- **navigateByUrl():** Simpler for absolute URLs, always treats as absolute path

5.5 Accessing Navigation State

Passing State:

```
// Sender Component  
export class UserListComponent {  
  constructor(private router: Router) {}  
  
  viewUserDetails(user: any) {  
    this.router.navigate(['/users', user.id], {  
      state: {  
        user: user,  
        returnUrl: '/users',  
        filters: { role: 'admin', active: true }  
      }  
    });  
  }  
}
```

Receiving State:

```
// Receiver Component  
export class UserDetailComponent implements OnInit {  
  user: any = null;  
  returnUrl: string = '/';  
  
  constructor(  
    private router: Router,
```

```
private route: ActivatedRoute
) {
  // Access navigation state
  const navigation = this.router.getCurrentNavigation();
  if (navigation?.extras.state) {
    this.user = navigation.extras.state['user'];
    this.returnUrl = navigation.extras.state['returnUrl'];

    console.log('Received user:', this.user);
    console.log('Return URL:', this.returnUrl);
  }
}

// Alternative: Access from window.history.state
ngOnInit() {
  if (window.history.state) {
    this.user = window.history.state.user;
    this.returnUrl = window.history.state.returnUrl || '/';
  }
}

goBack() {
  this.router.navigateByUrl(this.returnUrl);
}
}
```

User clicks on "John Doe" in user list

Navigation:

URL: /users/123

State: {

user: {id: 123, name: "John Doe", email: "john@example.com"},

returnUrl: "/users",

filters: {role: "admin", active: true}

}

Console Output:

Received user: {id: 123, name: "John Doe", ...}

Return URL: /users

✓ State passed without appearing in URL!

5.6 Real-World Example: Authentication Flow

Login Component:

```
import { Component } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-login',
  template: `
    <div class="login-form">
      <h2>Login</h2>
      <input [(ngModel)]="username" placeholder="Username">
      <input [(ngModel)]="password" type="password" placeholder="Password">
      <button (click)="login()">Login</button>
      <p *ngIf="errorMessage">{{ errorMessage }}</p>
    </div>
  `
})
export class LoginComponent implements OnInit {
  username = '';
  password = '';
  errorMessage = '';
  returnUrl = '/';

  constructor(
    private router: Router,
    private route: ActivatedRoute
  ) {}

  ngOnInit() {
    // Get return URL from query params
    this.route.queryParamMap.subscribe(params => {
      this.returnUrl = params.get('returnUrl') || '/';
      console.log('Return URL:', this.returnUrl);
    });
  }

  login() {
    // Simulate authentication
    if (this.username === 'admin' && this.password === 'password') {
      console.log('Login successful');

      // Navigate to return URL or default
      this.router.navigateByUrl(this.returnUrl);
    }
  }
}
```

```
    } else {  
      this.errorMessage = 'Invalid credentials';  
    }  
  }  
}
```

Auth Guard (Protecting Routes):

```
import { Injectable } from '@angular/core';  
import { Router, UrlTree } from '@angular/router';  
import { Observable } from 'rxjs';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class AuthGuard {  
  constructor(private router: Router) {}  
  
  canActivate(  
    route: ActivatedRouteSnapshot,  
    state: RouterStateSnapshot  
  ): boolean | UrlTree {  
    const isAuthenticated = this.checkAuth();  
  
    if (isAuthenticated) {  
      return true;  
    }  
  
    // Redirect to login with return URL  
    console.log('Not authenticated, redirecting to login');  
    console.log('Attempted URL:', state.url);  
  
    return this.router.createUrlTree(['/login'], {  
      queryParams: { returnUrl: state.url }  
    });  
  }  
  
  checkAuth(): boolean {  
    // Check if user is authenticated  
    return false; // Simulate not authenticated  
  }  
}
```

Flow Example:

1. User tries to access: /dashboard
2. Auth Guard checks authentication: FAIL
3. Redirect to: /login?returnUrl=/dashboard
4. User enters credentials and clicks Login
5. After successful login, navigate to: /dashboard

Console Output:

Not authenticated, redirecting to login
Attempted URL: /dashboard
Return URL: /dashboard
Login successful
→ Navigating to /dashboard

6. Best Practices

6.1 Child Routes Best Practices

Do:

- Use child routes for related functionality
- Provide default child route with empty path
- Keep route hierarchy shallow (2-3 levels max)
- Use meaningful route paths
- Place common layout logic in parent component

Don't:

- Don't create overly deep nesting (4+ levels)

- Don't forget `<router-outlet>` in parent components
- Don't use child routes when simple routing would suffice

6.2 Route Parameters Best Practices

Do:

- Use `paramMap` observable for reactive components
- Use `snapshot.paramMap` only for one-time reads
- Validate parameter values before using them
- Handle missing or invalid parameters gracefully
- Unsubscribe from `paramMap` in `ngOnDestroy`

6.3 Query Parameters Best Practices

Do:

- Use query params for optional filters and settings
- Use `queryParamsHandling: 'merge'` to preserve existing params
- Provide default values for query parameters
- Keep query param names consistent across app
- Use `queryParams` observable for reactive updates

Don't:

- Don't use query params for required/critical data
- Don't pass sensitive data in query params (visible in URL)
- Don't create excessively long query strings

6.4 Navigation Best Practices

Do:

- Use `navigate()` for flexibility with route parameters
- Use `navigateByUrl()` for simple absolute URL navigation
- Handle navigation errors with promises or observables
- Use `replaceUrl: true` for redirect scenarios
- Pass sensitive data via navigation state, not URL

7. Practice Exercises

Exercise 1: Build Admin Dashboard with Child Routes

Create an admin dashboard with:

- Parent route: `/admin`
- Child routes: `users`, `products`, `orders`, `settings`
- Each child should have its own nested routes (list, create, edit)
- Implement breadcrumb navigation
- Use `routerLinkActive` for active menu highlighting

Exercise 2: Product Catalog with Filters

Build a product catalog that:

- Uses query parameters for category, price range, sort order
- Preserves filter state in URL (shareable links)
- Implements pagination with query params
- Reacts to query parameter changes without component re-initialization
- Shows "Clear Filters" button when filters are active

Exercise 3: Blog with Anchor Navigation

Create a blog article page with:

- Table of contents using fragments
- Smooth scrolling to sections
- Highlight active section in TOC
- Share buttons that include fragment in URL
- Back to top button

Exercise 4: Multi-Step Form with Route Parameters

Implement a multi-step registration form where:

- Each step is a child route: `/register/step1` , `/register/step2` , etc.
- Pass form data between steps using navigation state
- Show progress indicator based on current route
- Prevent navigation to future steps (route guards)
- Allow editing previous steps

Exercise 5: Search Results with Complex Routing

Build a search results page with:

- Search term in route parameter: `/search/:term`
- Filters in query parameters
- Selected result in fragment: `#result-123`
- Programmatic navigation between results
- Browser back/forward support

7. Complete Standalone Example

Here's a complete example showing advanced routing with standalone components:

User Management with Child Routes and Parameters

Main App Routes (app.routes.ts):

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { UsersComponent } from './users/users.component';
import { UserListComponent } from './users/user-list.component';
import { UserDetailComponent } from './users/user-detail.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'users',
    component: UsersComponent,
    children: [
      { path: '', component: UserListComponent },
      { path: ':id', component: UserDetailComponent }
    ]
  }
];
```

Users Parent Component:

```
// users.component.ts
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-users',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <div class="users-container">
      <h2>User Management</h2>
      <nav>
        <a routerLink="">All Users</a>
      </nav>
      <router-outlet></router-outlet>
    </div>
```

```

})
export class UsersComponent {}

```

User List Component:

```

// user-list.component.ts
import { Component, OnInit } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [CommonModule, FormsModule, RouterLink],
  template: `
    <div>
      <h3>User List</h3>

      <!-- Filters -->
      <div class="filters">
        <input [(ngModel)]="searchTerm"
              (ngModelChange)="onFilterChange()"
              placeholder="Search users...">
        <select [(ngModel)]="roleFilter" (change)="onFilterChange()">
          <option value="">All Roles</option>
          <option value="admin">Admin</option>
          <option value="user">User</option>
        </select>
      </div>

      <!-- User List -->
      <div *ngFor="let user of users" class="user-card">
        <a [routerLink]="[user.id]"
          [queryParams]="{returnFilters: true}">
          {{ user.name }} ({{ user.role }})
        </a>
      </div>
    </div>
  `
})
export class UserListComponent implements OnInit {
  users = [

```

```
{ id: 1, name: 'John Doe', role: 'admin' },
{ id: 2, name: 'Jane Smith', role: 'user' },
{ id: 3, name: 'Bob Johnson', role: 'admin' }
];

searchTerm = '';
roleFilter = '';

constructor(
  private router: Router,
  private route: ActivatedRoute
) {}

ngOnInit() {
  // Read query params on init
  this.route.queryParamMap.subscribe(params => {
    this.searchTerm = params.get('search') || '';
    this.roleFilter = params.get('role') || '';
  });
}

onFilterChange() {
  this.router.navigate([], {
    relativeTo: this.route,
    queryParams: {
      search: this.searchTerm || null,
      role: this.roleFilter || null
    },
    queryParamsHandling: 'merge'
  });
}
}
```

User Detail Component:

```
// user-detail.component.ts
import { Component, OnInit, OnDestroy } from '@angular/core';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-user-detail',
  standalone: true,
```

```

imports: [CommonModule, RouterLink],
template: `
  <div *ngIf="user">
    <h3>User Details</h3>
    <p><strong>ID:</strong> {{ user.id }}</p>
    <p><strong>Name:</strong> {{ user.name }}</p>
    <p><strong>Role:</strong> {{ user.role }}</p>

    <button (click)="goBack()">Back to List</button>
    <button (click)="editUser()">Edit User</button>
  </div>
`
})
export class UserDetailComponent implements OnInit, OnDestroy {
  user: any = null;
  userId: string | null = null;
  private destroy$ = new Subject<void>();

  constructor(
    private route: ActivatedRoute,
    private router: Router
  ) {}

  ngOnInit() {
    // React to parameter changes
    this.route.paramMap.pipe(
      takeUntil(this.destroy$)
    ).subscribe(params => {
      this.userId = params.get('id');
      this.loadUser(this.userId);
    });
  }

  loadUser(id: string | null) {
    console.log('Loading user:', id);
    // Simulate loading user
    this.user = {
      id: id,
      name: `User ${id}`,
      role: id === '1' ? 'admin' : 'user'
    };
  }

  goBack() {
    // Navigate back with filters preserved
    this.router.navigate(['../'], {
      relativeTo: this.route,

```

```
        queryParamsHandling: 'preserve'
    });
}

editUser() {
    this.router.navigate(['../edit', this.userId], {
        relativeTo: this.route,
        state: { user: this.user }
    });
}

ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
}
}
```

Bootstrap (main.ts):

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
    providers: [
        provideRouter(routes)
    ]
}).catch(err => console.error(err));
```

Standalone Benefits Demonstrated:

- No NgModules required
 - Direct component imports
 - Simpler project structure
 - Same routing functionality as module-based
 - Better tree-shaking and smaller bundles
-

8. Summary

In this module, you learned:

- **Child Routes:** Creating nested routing structures with multiple router-outlets
- **Route Parameters:** Dynamic URL segments, snapshot vs observable approach
- **Query Parameters:** Optional URL parameters for filters, sorting, and pagination
- **Fragments:** Hash-based navigation for scrolling to page sections
- **Programmatic Navigation:** Using Router service with `navigate()` and `navigateByUrl()`
- **Navigation Extras:** Passing state, handling query params, relative navigation
- **Best Practices:** When to use each routing technique and common patterns

Advanced routing enables you to build complex, feature-rich applications with intuitive navigation structures that enhance user experience and maintain application state through URLs.

Key Takeaway: Both module-based and standalone approaches support the same advanced routing features. Standalone components offer simpler syntax with less boilerplate while maintaining full routing functionality. The Router service and its APIs work identically in both approaches.

Next Module Preview: In Module 11.3, we'll explore Route Protection with Guards (`CanActivate`, `CanDeactivate`, `CanLoad`) and Resolvers for pre-fetching data before route activation.

Module 11.2 - Advanced Routing | Angular Framework Training